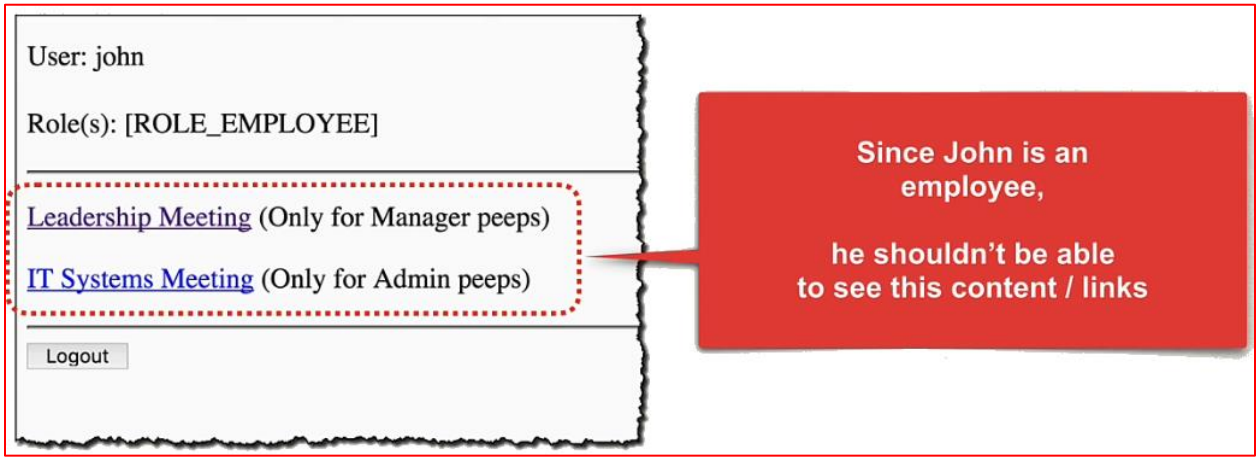
Spring MVC Security - Display Content Based on Roles

In this lesson, we'll explore how to conditionally display content based on user roles using Spring Security. Let's dive in!

Consider our familiar homepage. Why display certain links to all users? For instance, John, who is an employee, shouldn't see links related to leadership or IT systems meetings. Even though our security configurations have already restricted direct access to those URLs, it's ideal not to display them to users without the necessary roles.



Objective: Display content selectively, depending on a user's role.

Let's take Mary as an example. She holds roles as both an employee and a manager. Thus, she should be able to see manager-specific content. To achieve this, we use Spring Security tags. Here's a snippet for context:

```
<security:authorize access="hasRole('MANAGER')">
    <!-- Content here is specific for managers -->
</security:authorize>
```



Any content within this tag is rendered exclusively for users with the `MANAGER` role. One might think, "What if a tech-savvy user tries to view the source?" Well, they won't find anything because the content isn't merely hidden; it's excluded from the generated HTML.

Continuing, let's discuss Susan, who is both an administrator and an employee. She should see admin-specific content, which we can delineate with a similar tag:

```
<security:authorize access="hasRole('ADMIN')">
    <!-- Content here is specific for administrators -->
</security:authorize>
```

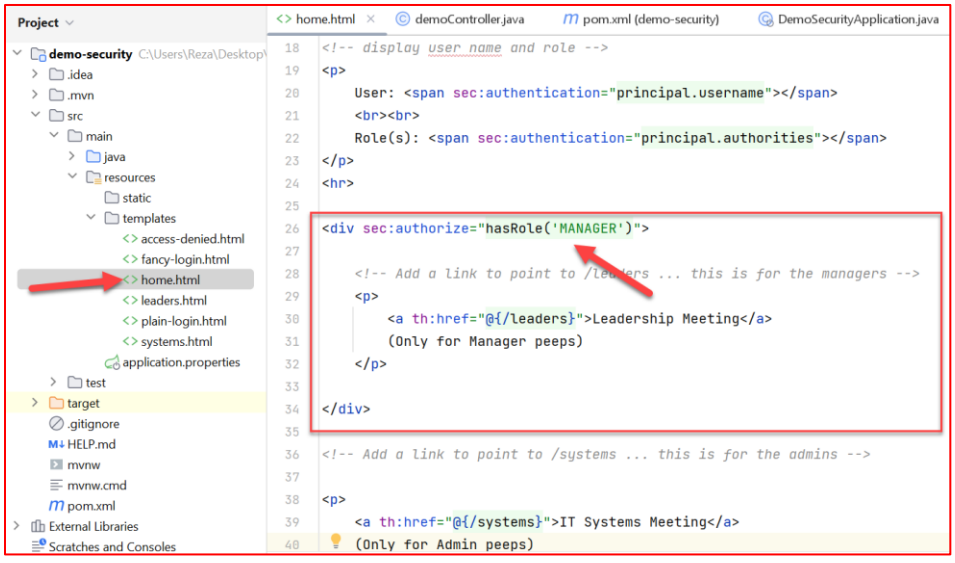


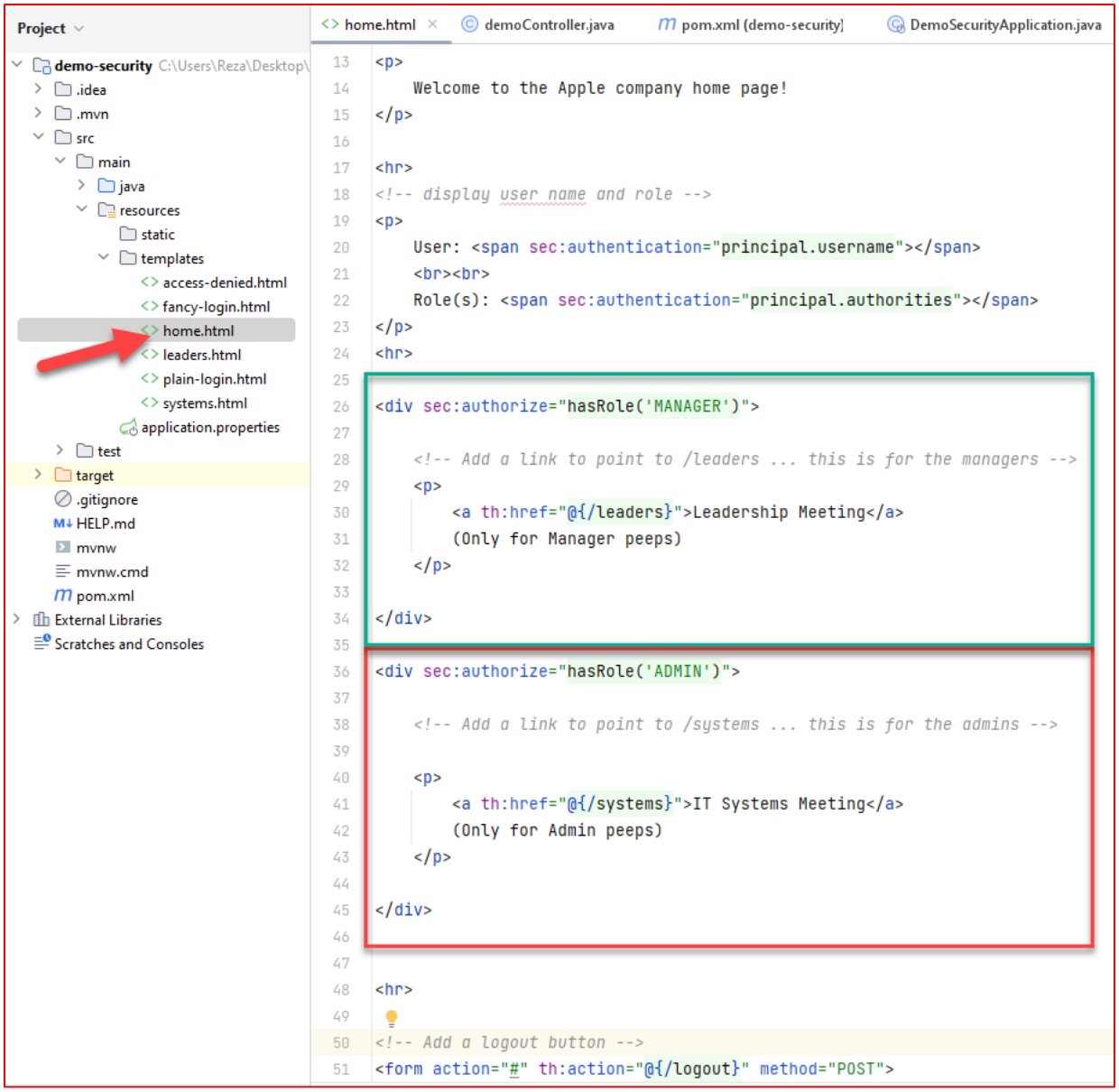
Exercise: We'll further customize our existing project. Let's run the application to visualize our changes. Logging in as John, he shouldn't see manager or admin-specific content. Our next step is to refine our homepage by embedding the security tags appropriately.

Be cautious, even with comments! We don't want to give away any unintended information.

Moving along, as we inspect the content, our goal is twofold:

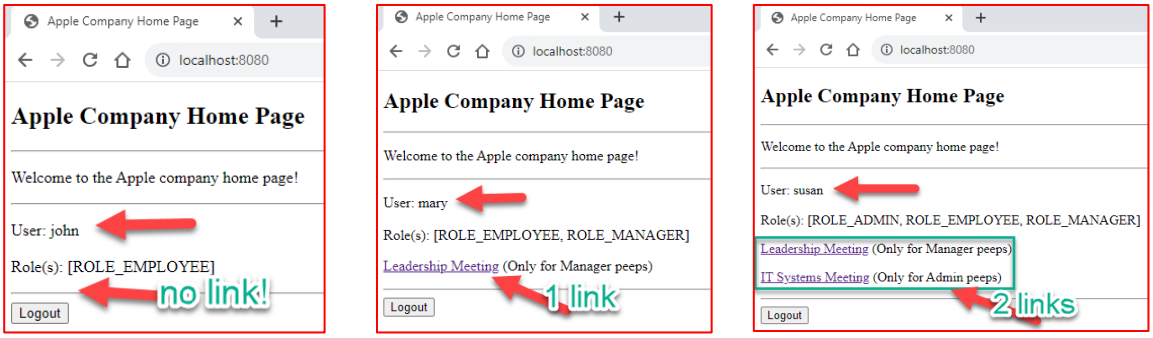
1. Display manager-specific content only to users with the `MANAGER` role.
2. Display admin-specific content only to those with the `ADMIN` role.





With the setup complete; Testing time! Let's check our implementation.

After refreshing the browser, the page displayed for `John` should omit the manager and admin-specific content. For `Mary`, we should see content relevant to her manager role. Lastly, testing with `Susan` should display the admin-specific content as expected.



As a bonus test, let's don the hacker hat. If John tries accessing URLs he heard about, like `/leaders` or `/systems`, he should be denied access. That's our security configurations at work, safeguarding our application.



In conclusion, not only have we locked down URLs based on roles, but we've also tailored the visible content to user roles, enhancing both security and user experience.